



TERMÉSZETTUDOMÁNYI  
KAR

# AI-Based Road User Detection Using Traffic- Monitoring Radar

Ármin Ihász, Gábor Tompos

---



BME

**To be presented on  
27/05/2026**



New model is currently being trained on Kaggle

Several models have been built, but the peak MAP@50 achieved was only ~0.34.

Hopefully we will be able to present the results of the new model.



## Goal

Objective: Detect Pedestrians, Cars, and Cyclists in real-time ( $< 40\text{ms}$ ) using a sparse 24 GHz pulse-Doppler radar.

### The Challenges:

- Extreme sparsity of radar point clouds (especially for pedestrians).
- Class imbalance (cars dominate the reflections).
- Camera-to-Radar BEV projection drift at long distances.



## Annotation & Post-Processing (Camera)

Inference Setup: Used YOLOv11m on camera footage with a low confidence threshold ( $\text{conf}=0.10$ ) to ensure high recall for heavily occluded pedestrians.

Tracking: Used ByteTrack for fast, reliable object ID consistency across frames.

Bike Merge: Applied custom post-processing to merge intersecting pedestrian + bicycle bounding boxes into a unified cyclist class, matching the physical reality of the scene.



## Radar Grid Mapping (Preprocessing)

Temporal Accumulation: Merged 10 consecutive frames (400ms window) to build dense physical contours out of sparse point clouds.

Spatial ROI: Strictly constrained the 70mx20m roadway area.

Physics-Based Channel Encoding:

Ch0 (Red): Occupancy / Point Density.

Ch1 (Green): Normalized Amplitude (compensated by  $R^2$  to offset the radar range degradation equation).

Ch2 (Blue): Skewed Doppler velocity (square-root scaling to highlight slow-moving pedestrians).

Cell Propagation: Applied spatial dilation to expand points into visible blobs for the CNN.



## Dataset Preparation

The Problem: Camera homography drifts at long distances, misaligning boxes from radar points.

The Solution: Implemented a proximity search (1.5m radius). Bounding boxes were snapped to the median of the radar points, ensuring perfect alignment.

Ghost Filtering: If a camera bounding box had zero supporting radar points, it was deleted to prevent YOLO from learning "empty space."

Target Biasing: Heavily oversampled (x3) the complex May 6th target domain to force the network to prioritize the benchmark environment.



## Network Architecture & Training Config

Architecture: Chosen YOLOv11n (Nano): To maintain inference speed well under the 40ms constraint and prevent overfitting on sparse radar graphics.

Hyperparameter Tuning: Switched to AdamW optimizer with a strictly tuned learning rate ( $lr_0=0.002$ ) and Cosine Annealing to stabilize gradient descent.

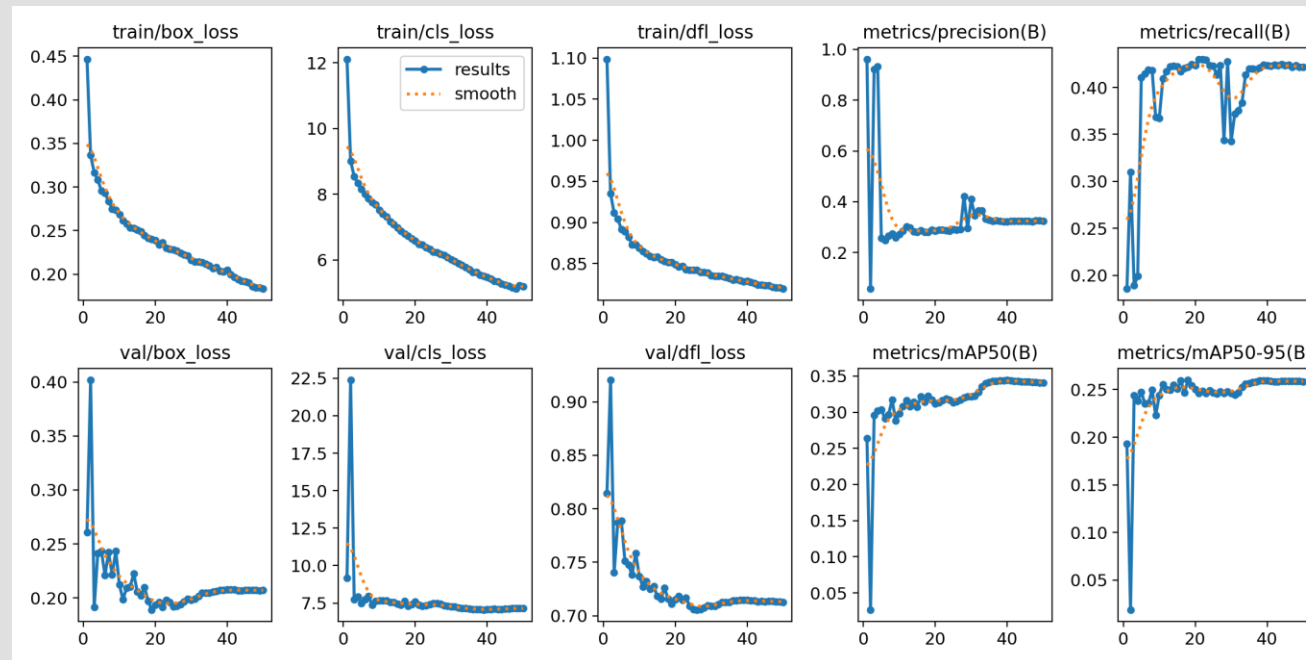
Augmentation Restraints: Disabled scaling, translation, and vertical flips. (Scaling alters the strict 0.1m/px physical rule, and vertical flips break the approaching/receding Doppler physics). Loss Weights: Adjusted  $cls=0.5$  and  $box=7.5$  to prioritize tight bounding-box regression over class-guessing.



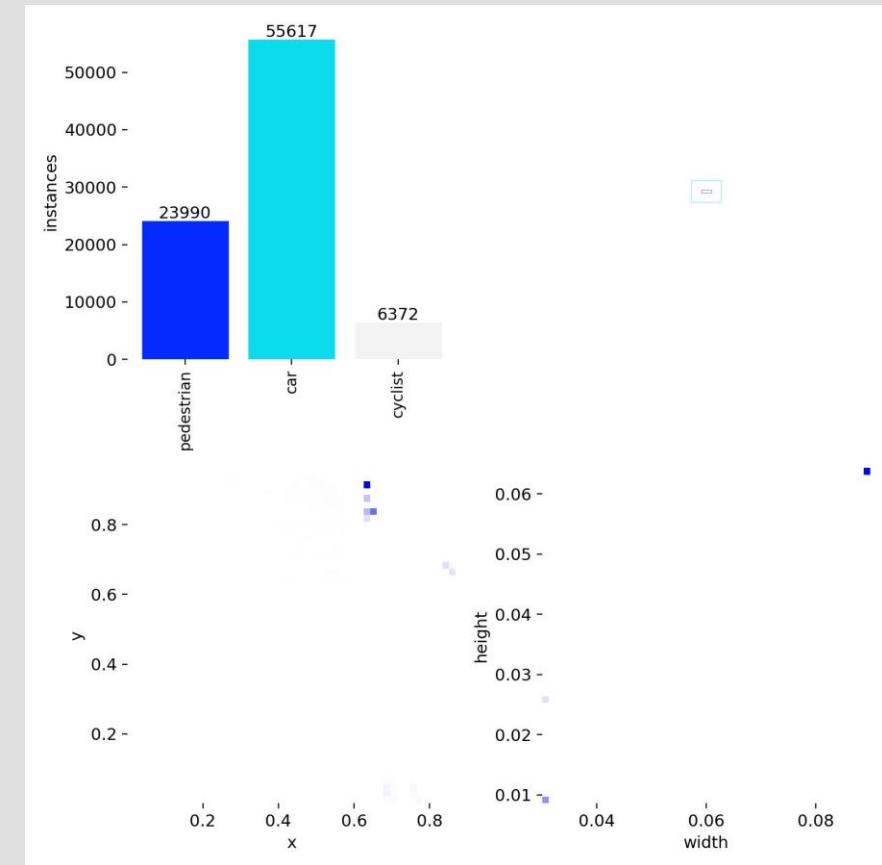
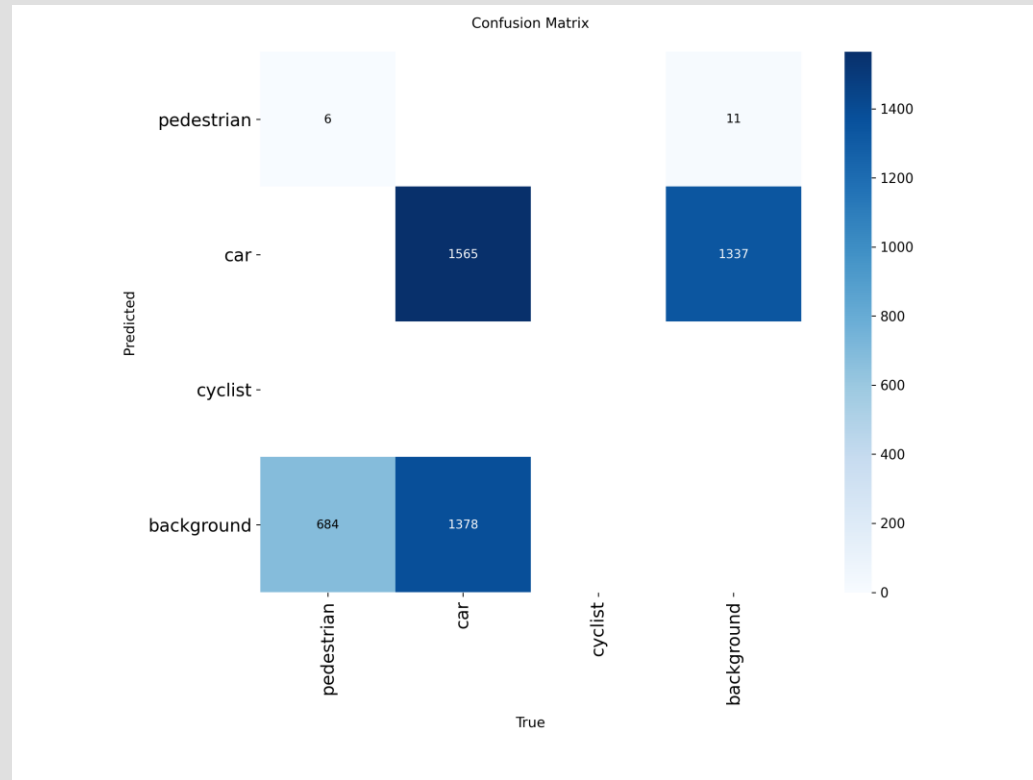
## Training & Convergence

Current Status: The model is currently finalizing its 60-epoch training on a Kaggle T4 dual-GPU cluster.

Initially tried 2 stage training, but then the 0.34mAP@50 after first stage drastically fell off



## Training & Convergence



## Finishing touches

Model Export: Converting the final PyTorch weights to an optimized ONNX graph.

Live Feeder Prep: Packaging the `grid_mapping_pipeline.py` into a callable module.

TensorRT Deployment for the live 40ms simulation

